Bare-Metal Architectural Audit: Fast-Qwen Engine

Hardware Target: Dual Xeon E5-2680 v4 (Broadwell) + NVIDIA GTX 1080 (Pascal GP104)
Software Paradigm: Zero-abstraction C++/CUDA / Hybrid MoE

This report compiles the critical failures across the toolchain, memory topology, and kernel logic, categorized by their impact on the "Low-Level Optimization Manual" specifications.

I. Toolchain & Handoff Annihilation

The space between the Python scripts and the C++ engine is currently a "mathematical void" where data layout and types are systematically shredded.

1. Structural Sequence Misalignment

- **Manual Spec (6.1):** The converter must generate a JSON manifest detailing exact byte offsets for the NUMAPool to ingest.
- **The Bug:** `convert_gguf.py` writes tensors (`attn_qkv` $\rightarrow$ `attn_k` $\rightarrow$ `attn_v` ...) while `main.cpp` reads them in a completely different order (`shared_gate` $\rightarrow$ `shared_up` ...).
- **Impact:** The engine executes `fread` of 3-bit MoE expert weights into pointers intended for FP16 GPU Attention tensors. Every pointer in the architecture contains the wrong data.

2. IEEE-754 Hardware Castration (FP16 $\rightarrow$ FP32)

- **The Bug:** The converter exports the backbone as `np.float16` , but CUDA kernels (`fused_attention_rotor.cu`) define the input as `const float*` (32-bit).
- **Hardware Reality:** The Pascal GPU reads 4 contiguous bytes for an FP32 register. It swallows two 2-byte FP16 numbers, fusing them. This forces the sign and exponent bits of the second number into the mantissa of the first.
- **Impact:** Resulting values are NaN/Inf cascades; matrix multiplications evaluate to garbage instantly.

II. CPU Pipeline & NUMA "Fortress" Failures

The engine currently violates every principle of the Broadwell-EP micro-architecture and dual-socket memory topology.

1. Broadwell L2 "Cache Suicide"

- **Manual Spec (3.1/3.2):** Prefetching must pull data into $L1/L2$ *just before* execution, using SPSC ring buffers to throttle the "look-ahead" to 16MB.

- **The Bug:** `worker_pool.cpp` uses `_MM_HINT_T1` in a tight loop over 16MB.
- **Hardware Reality:** The Xeon E5 v4 has only 256KB of L2 per core. Firing `T1` over 16MB overwrites the entire L2 cache 64 times in a fraction of a millisecond, flushing the stack, activations, and hot variables needed for the math.
- **Impact:** A prefetch optimization is converted into a weapon that starves the execution units.

2. NUMA Node-0 Sabotage

- **Manual Spec (2.1):** Hard-lock the process to CPU 0 (Affinity 0x3FFF) to avoid the ~100ns QPI latency penalty of crossing sockets.
- **The Bug:** `lock_process_to_numa0()` grabs the *System* Affinity Mask (all cores) and applies it to the process.
- **Impact:** This actively *unlocks* the threads to span both NUMA nodes, ensuring maximum memory latency and slamming into the QPI link bandwidth limit.

3. The 8KB Lambda "Heap Bomb"

- **The Bug:** Orchestrator captures `ExpertWeights` structs (which contain `float rms_weight[2048]`) by value in `std::function` closures.
- **Hardware Reality:** Small Object Optimization (SOO) in `std::function` usually caps at 32 bytes. Capturing 8KB forces a silent `new` allocation on the heap for every expert dispatch (360 allocations per token).
- **Impact:** OS memory allocator locks become the primary performance bottleneck of the entire inference loop.

III. GPU Architecture & Pascal Kernel Failures

The GPU kernels ignore the GP104 silicon's specific limitations (FP16 throttling and SFU count).

1. The Attention Serial Death Loop

- **Location:** `fused_attention_rotor.cu`
- **The Bug:** The dot product is not parallelized across the warp. Every thread in the block computes the same 128-iteration serial loop.
- **Impact:** Thread 0 does all work while 1-31 execute redundantly; inflates execution time by 128x.

2. 48-Byte Alignment Erasure

- **Manual Spec (4.1):** Kernels must load 128-bit chunks (`LDG.E.128`) to minimize memory latency.
- **The Bug:** `BlockQ4` is 36 bytes. Using `alignas(16)` pads it to 48 bytes.
- **Impact:** A 48-byte struct cannot be loaded via a vectorized 128-bit instruction. The compiler serializes the load into fragmented 32-bit reads, permanently bottlenecking the GTX 1080's VRAM bandwidth.

3. Taylor Expansion Lie (The SFU Bottleneck)

- **Manual Spec (5.2):** Replace SSM Softplus ($log(1 + e^x)$) with a rational polynomial Taylor approximation to utilize the 128 FMA cores per SM.
- **The Bug:** The code uses standard `silu` with `expf(-x)` , which is an SFU-bound instruction.
- **Hardware Reality:** Pascal has only 32 SFUs per SM. Forcing 8192-dimension projections through them creates a massive pipeline stall.

IV. Algorithmic & Mathematical Logic Failures

1. The QuIP# Shift-Mask Vulnerability

- **The Bug:** `memcpy` is used to pull 3 bytes (24 bits) into a 32-bit integer (`uint32_t val32`).
- **The Reality:** On little-endian x86_64, the 4th byte of `val32` remains uninitialized garbage. Because the AVX2 logic uses logical right shifts (`val32 >> (21 - i*3)`), this garbage shifts down into the active mask.
- **Impact:** Floating-point weights are polluted with random memory artifacts.

2. Asymmetric V-Cache Destruction

- **Manual Spec (4.2):** Use Symmetric 4-bit Rotor Quantization for both K and V caches.
- **The Bug:** Implementation uses 4-bit for K, but shifts V to pack 4 values per byte (2-bit precision).
- **Impact:** Catastrophically destroys the model's sequence recall; the "Quaternion Sandwich" cannot save 2-bit precision data.

3. RoPE Warp Cutoff

- **The Bug:** `apply_rope` limits execution to `tid < 32` . This applies positional context to only 64 dimensions.
- **Impact:** Since `HEAD_DIM` is 128, the second half of every attention vector receives zero positional context.

V. Orchestrator & CLI Asphyxiation

1. The BPE Tokenization Shredder

- **The Bug:** `chat.py` calls `tokenizer.decode([int(buffer)])` inside a byte-by-byte loop.
- **Reality:** Qwen uses BPE. A single multi-byte UTF-8 character (like a Kanji) is split across multiple tokens. Decoding them in isolation returns replacement characters (``) or fractured text.

2. Unhandled Segfault Deadlock

- **The Bug:** The Python main thread blocks on `proc.stdout.read(1)` while the stderr thread logs diagnostics.
- **Impact:** Given the memory misalignment and null pointer dereferences (`W->hw` is never allocated), a segfault is guaranteed. When the process dies, the OS doesn't flush the buffer. The Python script hangs infinitely waiting for a newline from a dead process.

VI. Implementation Status vs. Optimization Manual

Feature	Manual Spec	Current State	Violation
NUMA Isolation	Set Affinity 0x3FFF	System Mask (0xFFFF)	**Critical**
Memory Pages	2MB Large Pages	Standard 4KB Pages	**Performance**
Weights (GPU)	Q4 (4-bit)	FP16/FP32 Hybrid	**OOM Hazard**
Weights (CPU)	Q3 (3-bit) QuIP#	Scalar Unpack/Garbage Mask	**Math Failure**
SRAM Convolution	Shared Memory Shift	Global VRAM Read/Write	**Bottleneck**
Guard Pages	Sequential Reservation	Separate Allocations	**Security/Stability**